

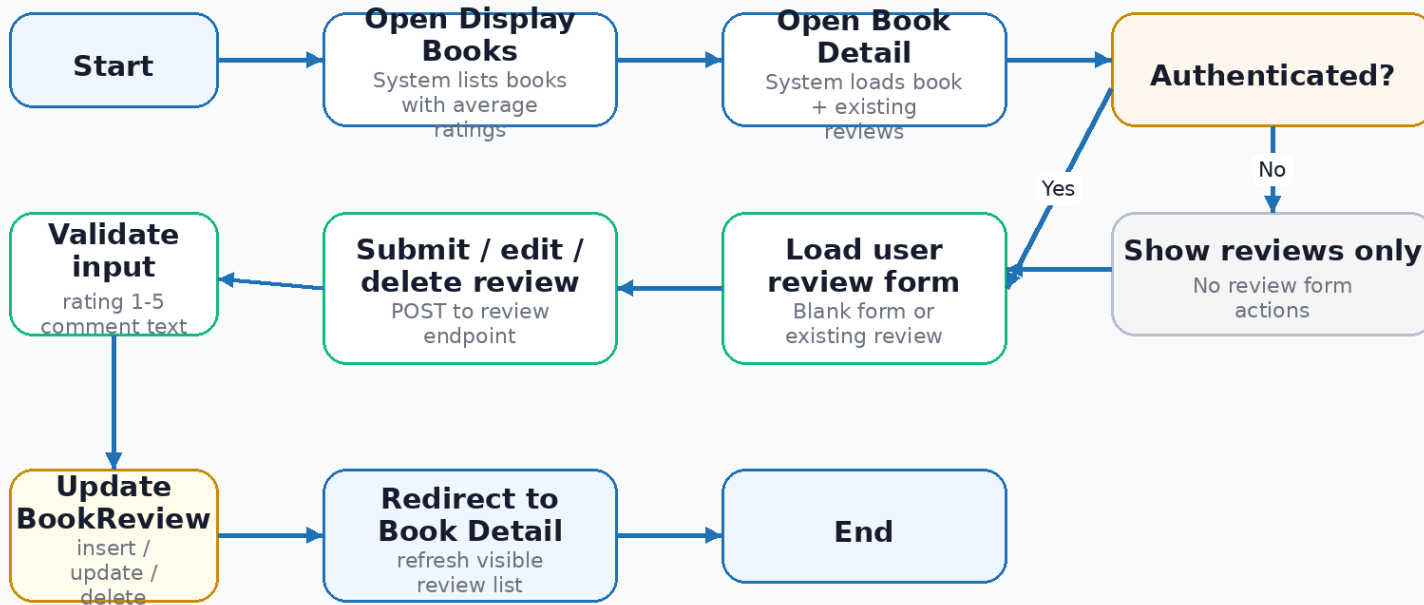
BookEx Requirement Documents

1. Requirement Document - Activity Diagram A

This activity diagram captures the catalog browsing and review workflow. It starts at the catalog, moves into book detail, checks authentication, and then branches into read-only behavior or review submission/edit/delete behavior.

Activity Diagram A

Browse catalog and create/edit a review



Generated from repository structure and source code review

Figure: 1. Requirement Document - Activity Diagram A

2. Requirement Document - Activity Diagram B

This activity diagram captures the private messaging workflow. It shows inbox/compose access, recipient selection, validation checks, thread creation or retrieval, message creation, and message reading.

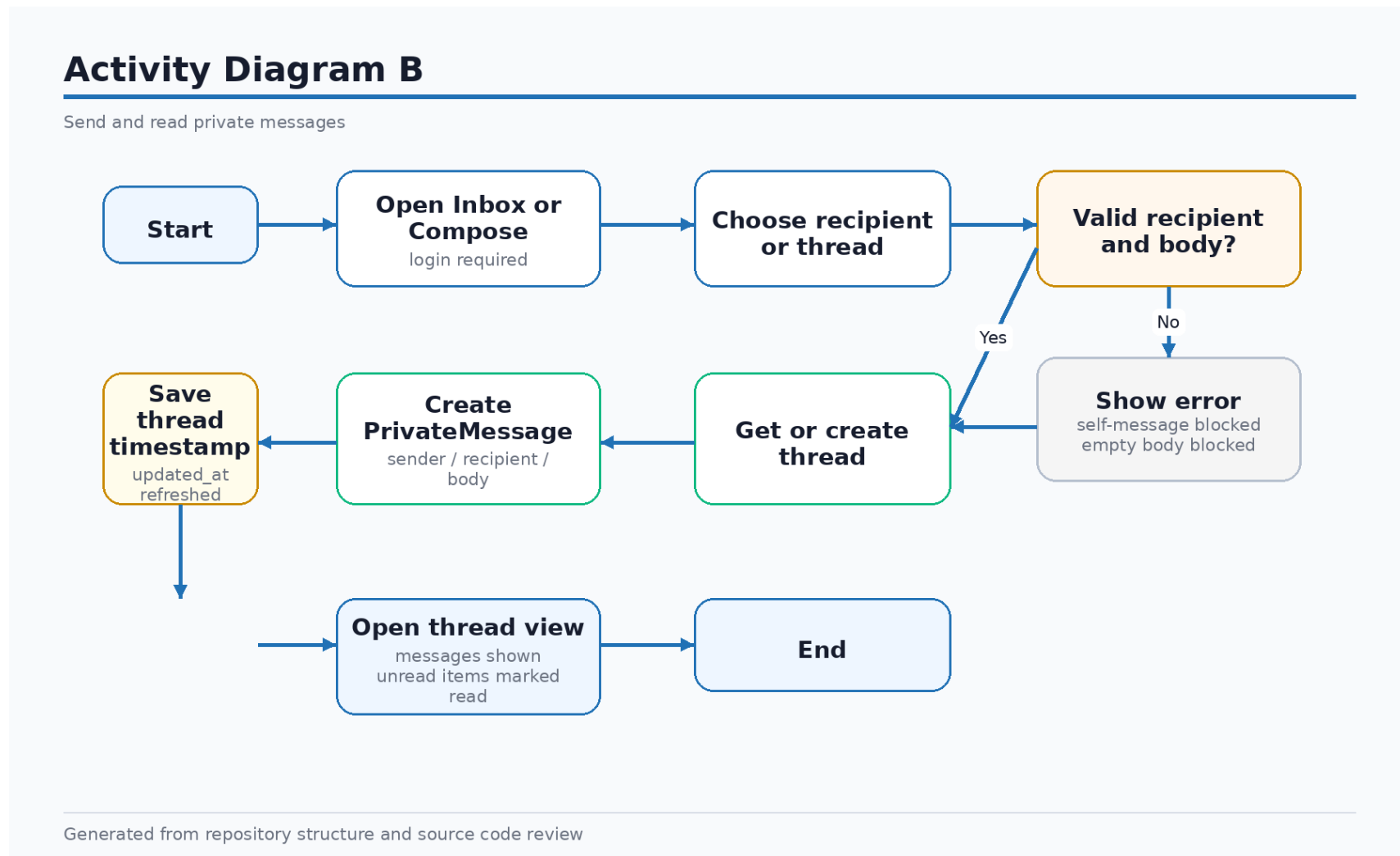


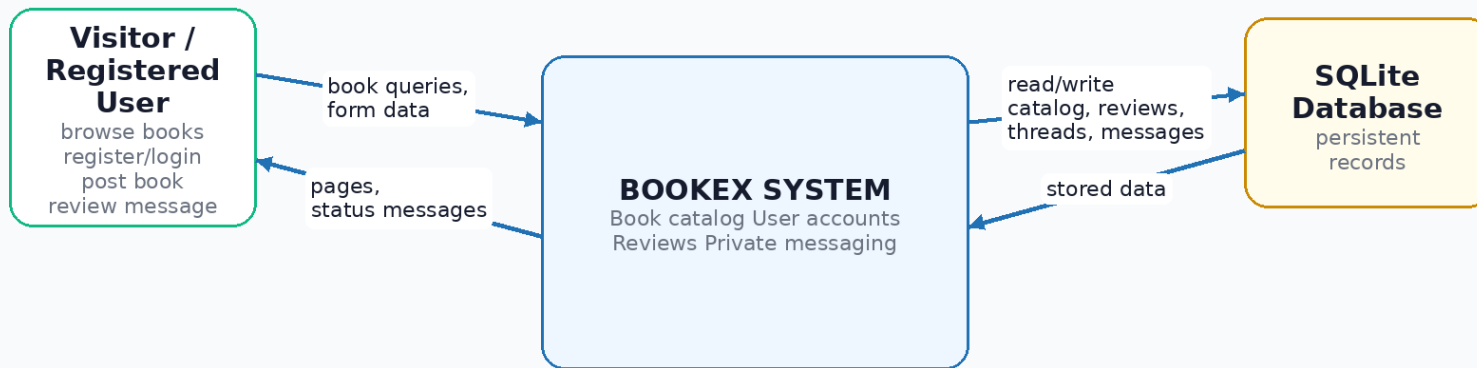
Figure: 2. Requirement Document - Activity Diagram B

3. Requirement Document - Data Flow Diagram (Context)

The context-level DFD treats BookEx as a single process. It exchanges requests and pages with users and reads/writes persistent data in SQLite.

Data Flow Diagram (Context)

System shown as a single process



Generated from repository structure and source code review

Figure: 3. Requirement Document - Data Flow Diagram (Context)

4. Requirement Document - Data Flow Diagram (Level 1)

The level 1 DFD decomposes the system into four major processes: browse catalog, post book, manage reviews, and messaging. These processes interact with the core data stores.

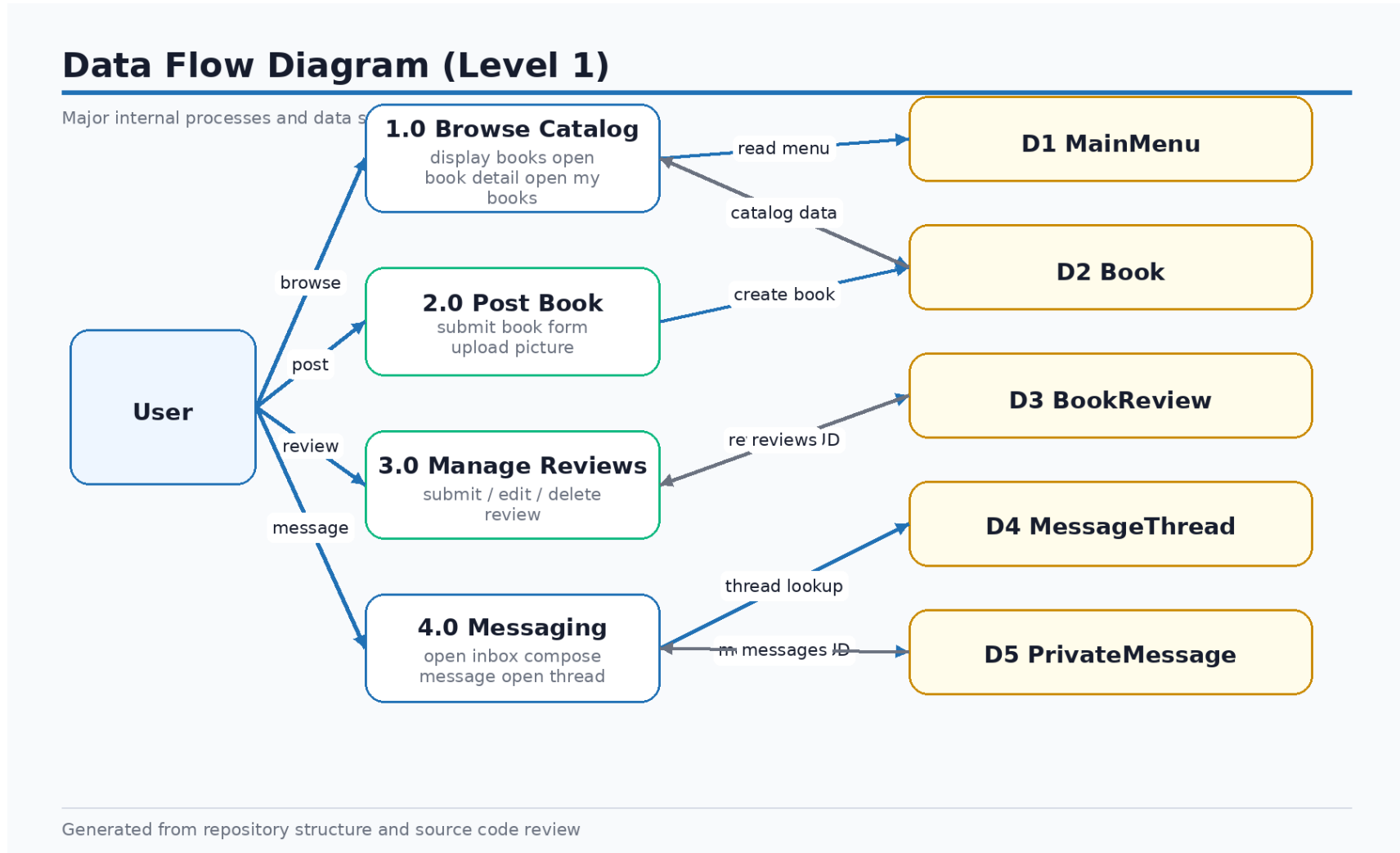


Figure: 4. Requirement Document - Data Flow Diagram (Level 1)

5. Requirement Document - Entity-Relationship Diagram

The ERD is inferred directly from the Django models. It highlights the user, book, review, thread, message, and menu structures along with the major foreign-key relationships.

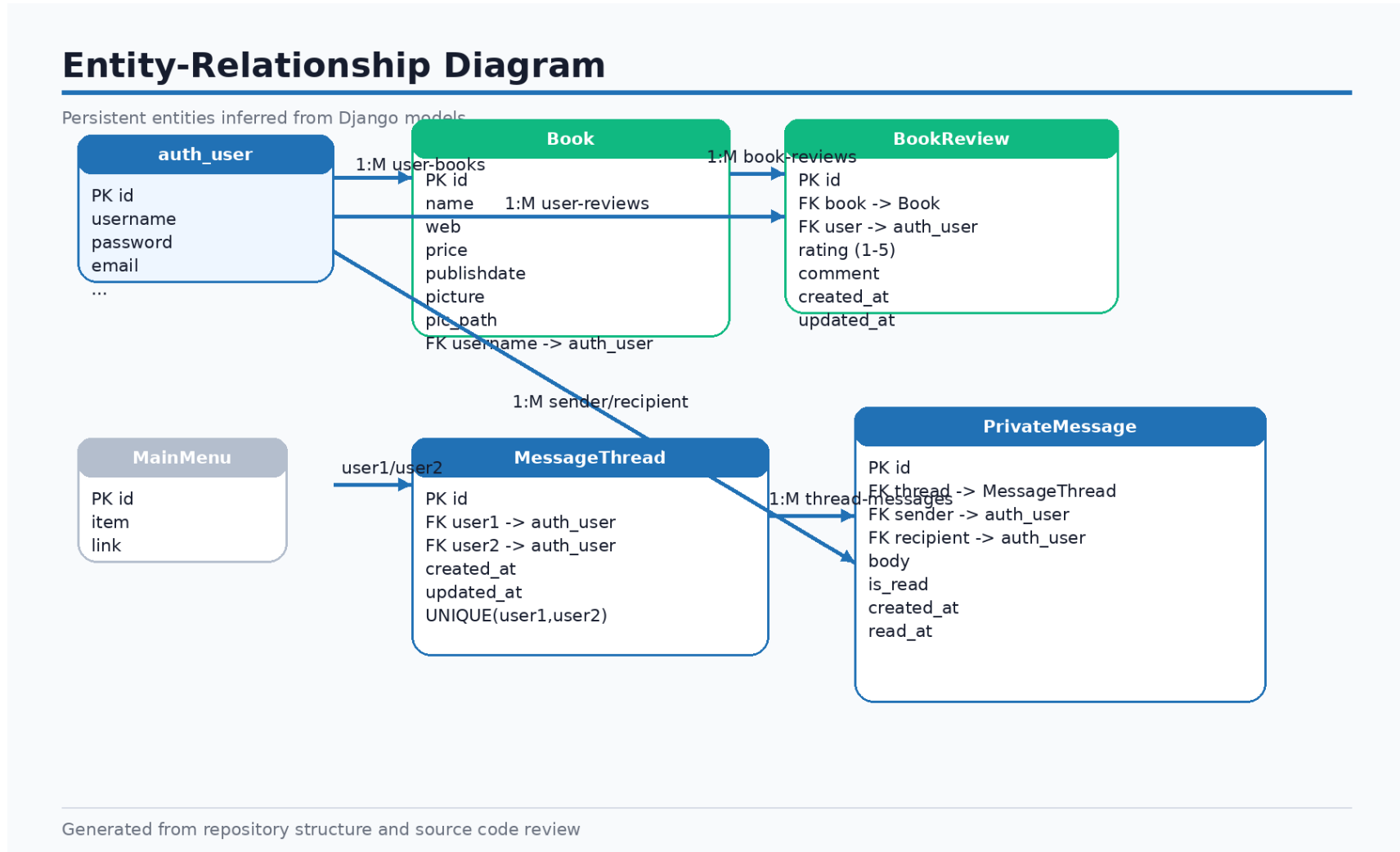


Figure: 5. Requirement Document - Entity-Relationship Diagram

6. Requirement Document - State Machine Diagram

The state machine focuses on the private-message lifecycle because the repository contains explicit unread and read behavior. A future deleted/archived state is shown as an optional extension rather than a currently implemented feature.

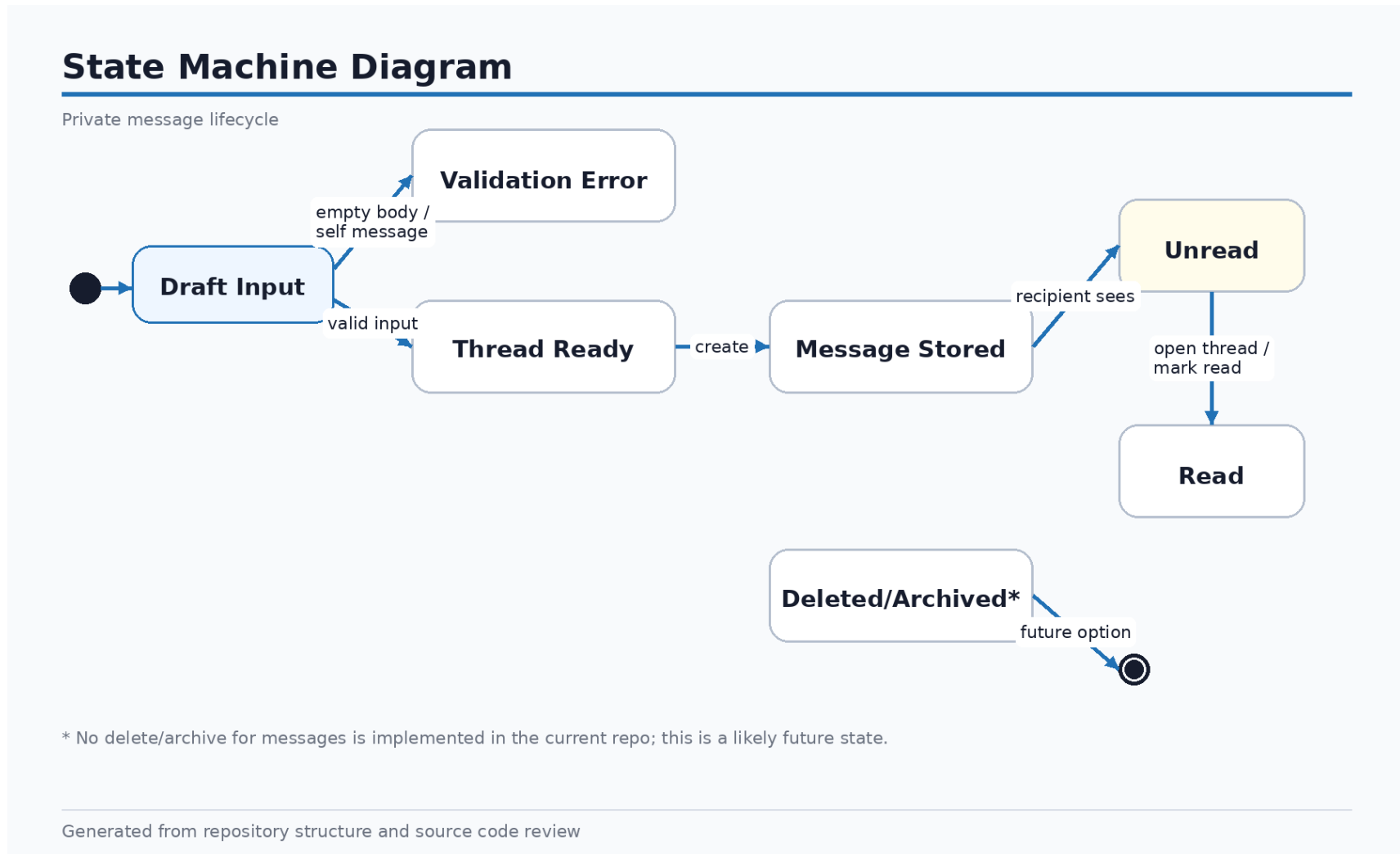


Figure: 6. Requirement Document - State Machine Diagram

7. Requirement Document - Component Diagram

This component diagram groups the implemented features into presentation, user account, catalog, review, messaging, persistence, and framework-service components.

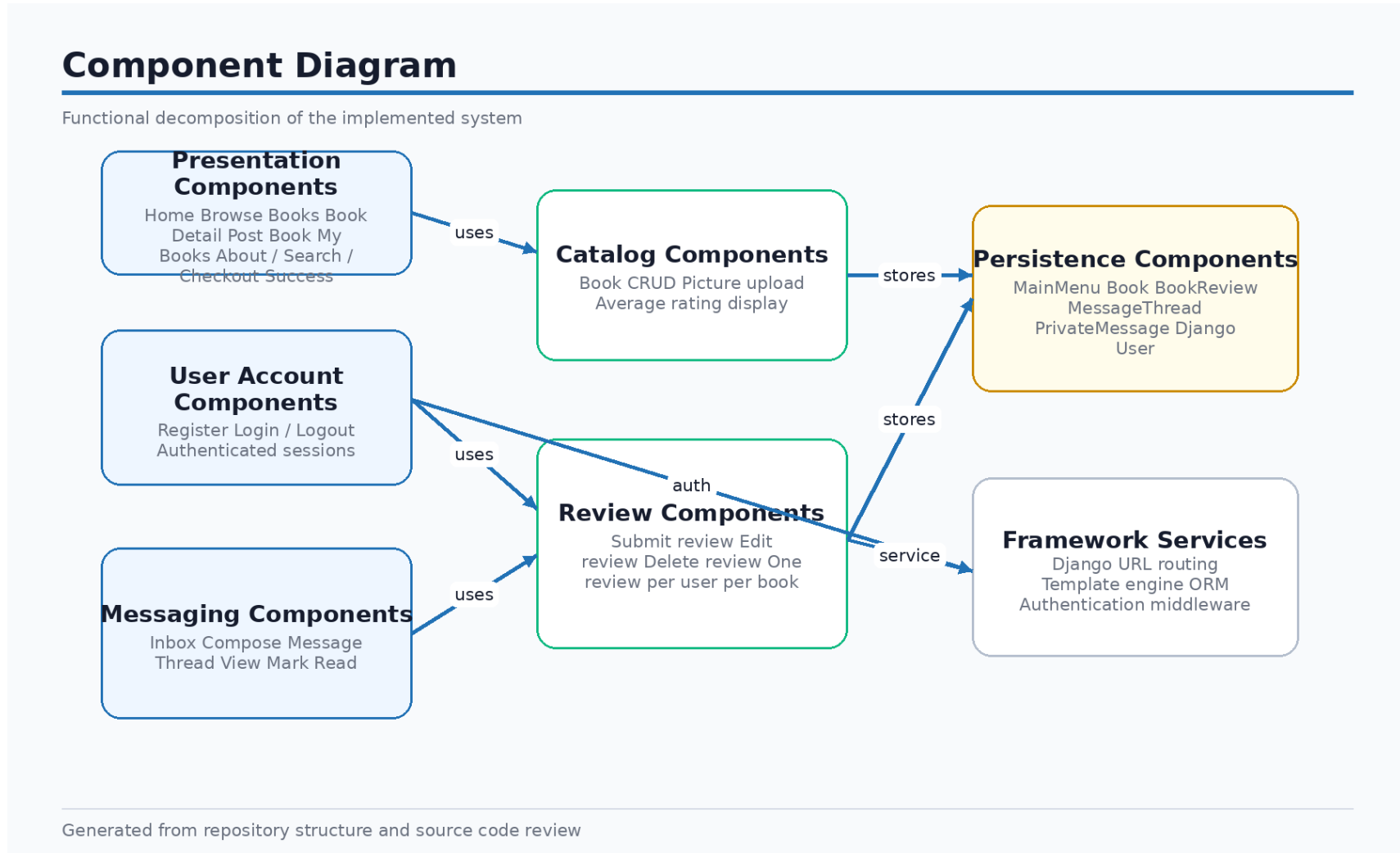


Figure: 7. Requirement Document - Component Diagram

8. Requirement Document - Data Dictionary

The following tables summarize the main persistent data structures visible in the repository.

MainMenu

Field	Type	Meaning	Notes
id	BigAutoField / PK	Unique row identifier	Created by Django
item	CharField(300)	Navigation label shown in menus	Unique
link	CharField(300)	Navigation target or route text	Unique

Book

Field	Type	Meaning	Notes
id	BigAutoField / PK	Unique book row identifier	Created by Django
name	CharField(200)	Book title/name	Required
web	URLField(300)	External or related URL for the book	Required
price	DecimalField(8,2)	Displayed book price	Required
publishdate	DateField(auto_now=True)	Date automatically refreshed on save	Auto-maintained
picture	FileField	Uploaded book image	Stored under static/uploads
pic_path	CharField(300)	Derived image path used for rendering	Editable=False; blank=True
username	ForeignKey(User)	Owner/uploader of the book	Nullable

BookReview

Field	Type	Meaning	Notes
id	BigAutoField / PK	Unique review row identifier	Created by Django
book	ForeignKey(Book)	Reviewed book	Required
user	ForeignKey(User)	Reviewer	Required
rating	PositiveSmallIntegerField	Score from 1 to 5	Validated 1-5
comment	TextField	Review comment text	Required
created_at	DateTimeField(auto_now_add=True)	Initial creation timestamp	Auto-maintained
updated_at	DateTimeField(auto_now=True)	Last update timestamp	Auto-maintained

MessageThread

Field	Type	Meaning	Notes
id	BigAutoField / PK	Unique thread identifier	Created by Django
user1	ForeignKey(User)	First participant	Required
user2	ForeignKey(User)	Second participant	Required
created_at	DateTimeField(auto_now_add=True)	Thread creation timestamp	Auto-maintained
updated_at	DateTimeField(auto_now=True)	Last message/update timestamp	Auto-maintained
UNIQUE(user1,user2)	Constraint	Only one thread per ordered user pair	Repository-enforced

PrivateMessage

Field	Type	Meaning	Notes
id	BigAutoField / PK	Unique message identifier	Created by Django
thread	ForeignKey(MessageThread)	Parent conversation thread	Required
sender	ForeignKey(User)	User sending the message	Required
recipient	ForeignKey(User)	User receiving the message	Required
body	TextField	Message content	Required
is_read	BooleanField	Read/unread status	Default=False
created_at	DateTimeField(auto_now_add=True)	Message creation time	Auto-maintained
read_at	DateTimeField	Timestamp when marked read	Nullable

auth_user (Django built-in)

Field	Type	Meaning	Notes
id	Integer / PK	User identifier	Built-in Django auth table
username	String	Login name referenced by the app	Used explicitly
password	String	Stored password hash	Built-in Django auth
email	String	Standard account email field	Framework field